OBJECT ORIENTED MODELLING & DESIGNING

REPORT

EDUFLOW

Academic Management & Productivity Portal

Course Code: 410241

Submitted by: Soniya Pawar
Roll No.: [Roll No]
Class / Division: [Class / Division]

Department: Computer
Academic Year: 2026-27

TITLE

EduFlow – Academic Management & Productivity Portal

Object Oriented Modelling and Design using UML

Problem Statement

Schools and colleges still manage academic work through notice boards, WhatsApp groups, emails and paper registers. Assignments are announced in one place, submitted in another and graded on paper, so students miss deadlines, teachers cannot see who has submitted, late and duplicate submissions are hard to track, and feedback reaches students late or not at all. Students also keep their own to-do lists separately, with no single view of everything that is due.

The aim of this project is to analyse and model EduFlow, a web-based Academic Management & Productivity Portal that brings these activities into one system. Admins must be able to manage users, classes, subjects and enrolment. Teachers must be able to publish assignments, review submissions, grade them, give feedback and return work for revision. Students must be able to view and submit assignments, manage personal tasks, see a calendar of deadlines, receive notifications and track their progress through charts. All data must be stored securely in a central MySQL database, and access must be controlled by role.

Objectives

- **Single platform:** assignments, submissions, grades, announcements and personal tasks in one place.
- **Accuracy:** record every submission with its time, attempt number and late status, so no work is lost or counted twice.
- **Timely feedback:** notify students automatically when work is published, received, graded or returned.
- **Productivity:** give students a dashboard, calendar and natural-language quick-add for personal tasks.
- **Security:** role-based access (Admin, Teacher, Student), hashed passwords, CSRF protection, login lockout and validated file uploads.

Scope

The system covers user and class management, assignment publishing, submission with file validation, grading and feedback, announcements, notifications and reminders, personal tasks, the calendar and progress analytics. Its users (actors) are the Admin, Teacher and Student. The Scheduler (a cron job) is a system actor that sends reminders, and the Mail Server is an external system used for password-reset emails.

Modelling Approach

The problem is modelled with UML in the following order: Class Diagram (static structure), State Model (life cycle of a submission), Activity Diagram (assignment workflow), Use Case Diagram (user functions), Component Diagram (software modules and interfaces), Sequence Diagram (object interaction when an assignment is submitted and graded) and finally the Deployment Diagram (physical architecture on which the modelled software runs). Each diagram is described on its own page, together with its key elements.

1. Class Diagram

A class diagram is a static structural UML diagram that shows the classes of a system, their attributes and operations, and the relationships between them.

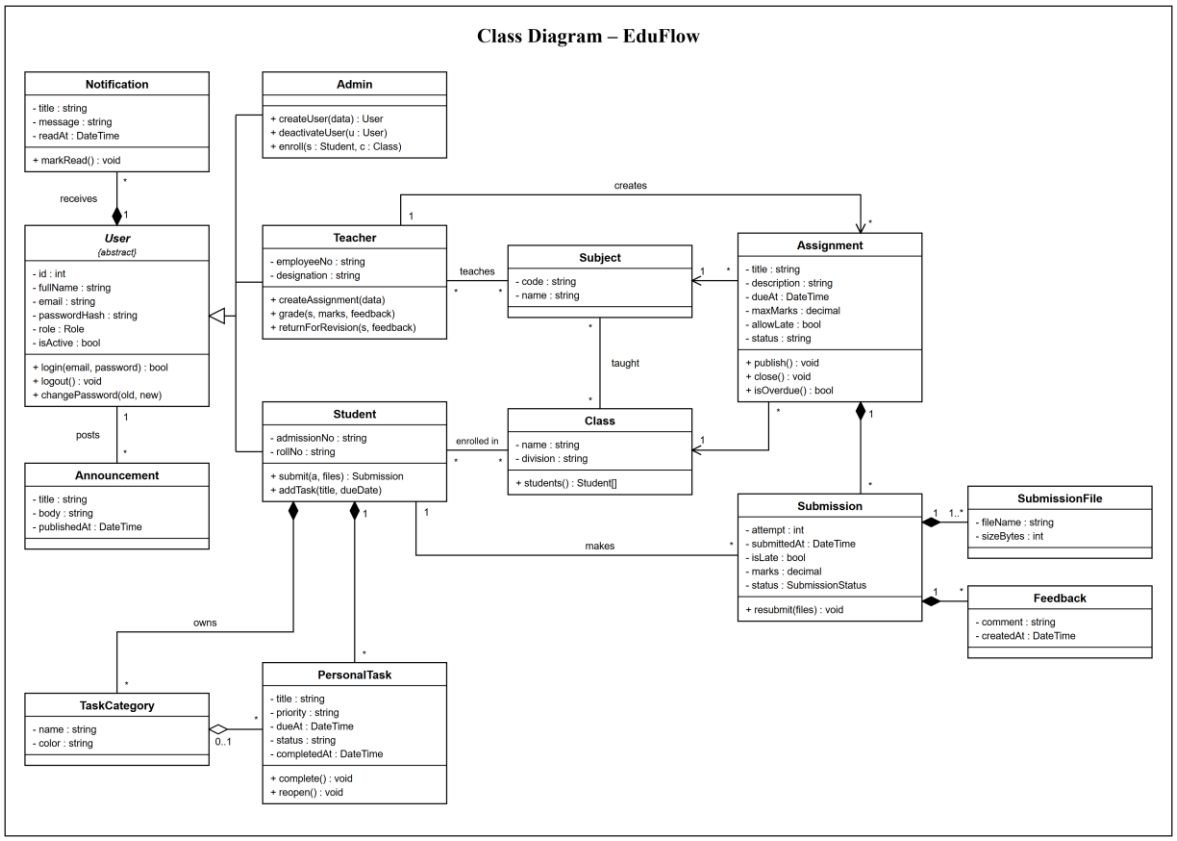


Figure 1: EduFlow – Class Diagram

Description

This diagram models the main classes of EduFlow. The abstract class User (id, fullName, email, passwordHash, role, isActive) provides login(), logout() and changePassword(). Admin, Teacher and Student inherit from User. Admin adds createUser(), deactivateUser() and enroll(); Teacher adds createAssignment(), grade() and returnForRevision(); Student adds submit() and addTask(). Every User receives Notifications (composition) and can post Announcements.

A Teacher teaches Subjects and a Student is enrolled in Classes (both many-to-many), and Subjects are taught in Classes. A Teacher creates zero or more Assignments; each Assignment (title, description, dueAt, maxMarks, allowLate, status) refers to exactly one Class and one Subject, shown by the navigable arrows, and supports publish(), close() and isOverdue().

An Assignment is composed of Submissions. A Student makes zero or more Submissions; each Submission (attempt, submittedAt, isLate, marks, status) is composed of one or more SubmissionFiles and zero or more Feedback entries, and supports resubmit(). A Student also owns PersonalTasks and TaskCategories, and a TaskCategory groups zero or more PersonalTasks (aggregation).

Key Elements

- **Classes:** User, Admin, Teacher, Student, Notification, Announcement, Subject, Class, Assignment, Submission, SubmissionFile, Feedback, PersonalTask and TaskCategory, each with attributes and operations.

- **Generalisation:** Admin, Teacher and Student inherit from the abstract class User (hollow-triangle arrow).
- **Composition (filled diamond):** User 1 to * Notification; Assignment 1 to * Submission; Submission 1 to 1..* SubmissionFile and 1 to * Feedback; Student 1 to * PersonalTask and TaskCategory.
- **Associations and multiplicity:** Teacher 1 to * Assignment (creates); Student 1 to * Submission (makes); Student * to * Class (enrolled in); Teacher * to * Subject (teaches); Subject * to * Class (taught); Assignment * to 1 Class and Subject.
- **Aggregation (hollow diamond):** TaskCategory 0..1 to * PersonalTask.
- **Visibility:** Attributes are private (-) and operations are public (+).

2. State Model (State Chart Diagram)

A state model describes the different states an object passes through and the transitions between them in response to events and conditions.

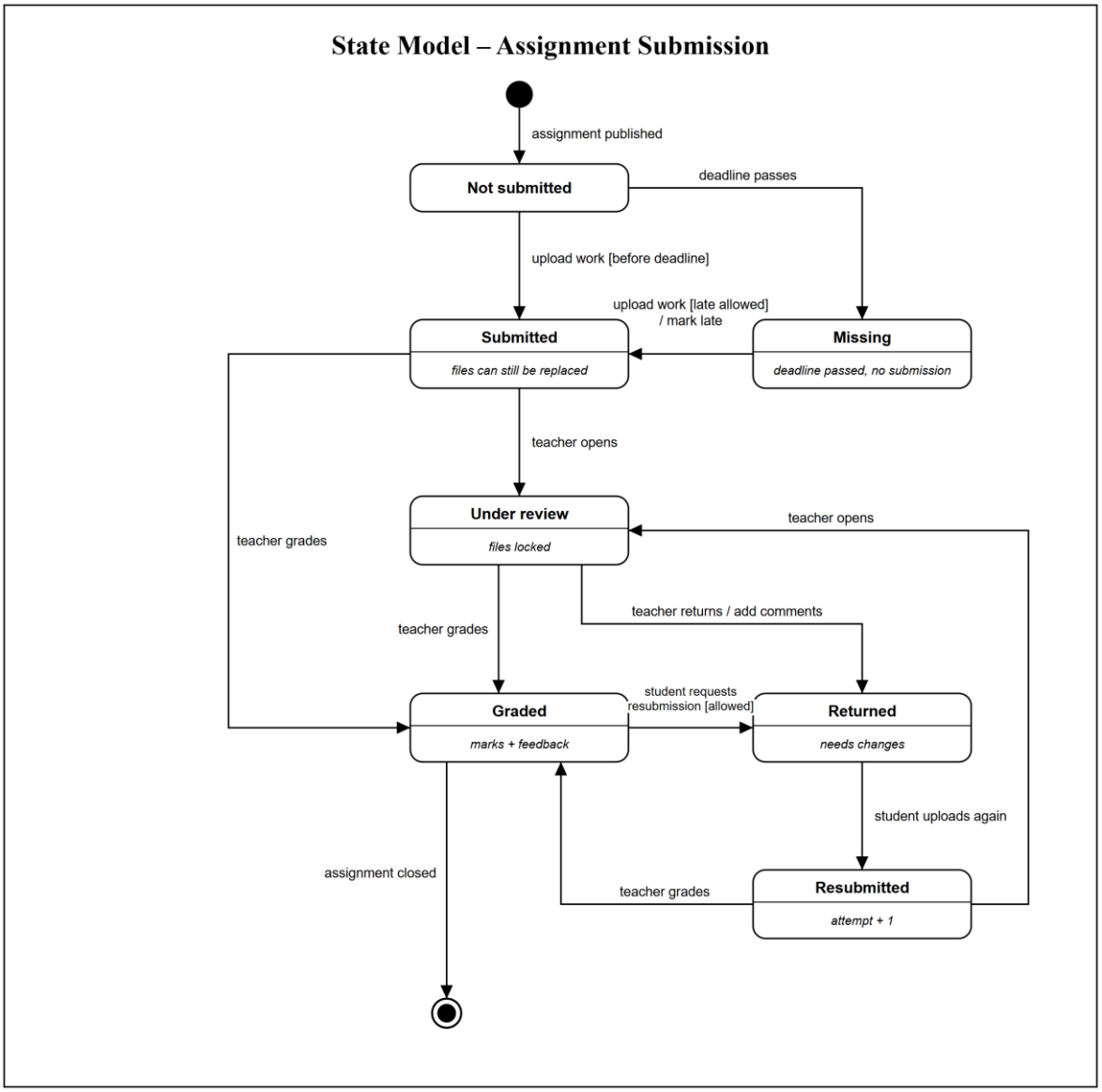


Figure 2: EduFlow – State Model (State Chart Diagram)

Description

This diagram shows the life cycle of a student's submission for one assignment. When the assignment is published the submission starts in the Not submitted state. If the student uploads work before the deadline it moves to Submitted, where the files can still be replaced. If the deadline passes first it moves to Missing; from there a late upload is accepted only when late submission is allowed, and the submission is marked late.

When the teacher opens a submission it moves to Under review and the files are locked. The teacher either grades it (Graded, with marks and feedback) or returns it for changes (Returned, with comments). A returned submission becomes Resubmitted when the student uploads again, which increases the attempt number, and it is then reviewed or graded again.

A graded submission can go back to Returned if the student requests a resubmission and this is allowed. When the assignment is closed the life cycle ends in the final state.

Key Elements

- **Initial and final states:** The filled circle starts the life cycle when the assignment is published; the circle with a ring ends it when the assignment is closed.
- **States:** Not submitted, Missing, Submitted, Under review, Graded, Returned and Resubmitted, each with an internal note (e.g. files locked, marks + feedback, attempt + 1).
- **Events:** upload work, deadline passes, teacher opens, teacher grades, teacher returns, student uploads again, student requests resubmission.
- **Guard conditions:** [before deadline], [late allowed] and [allowed] decide whether a transition may fire.
- **Actions:** / mark late and / add comments are performed during the transition.
- **Loop:** Returned → Resubmitted → Under review → Returned allows several revision cycles.

3. Activity Diagram

An activity diagram models the workflow of a process as a sequence of actions and decisions; swimlanes show who is responsible for each action.

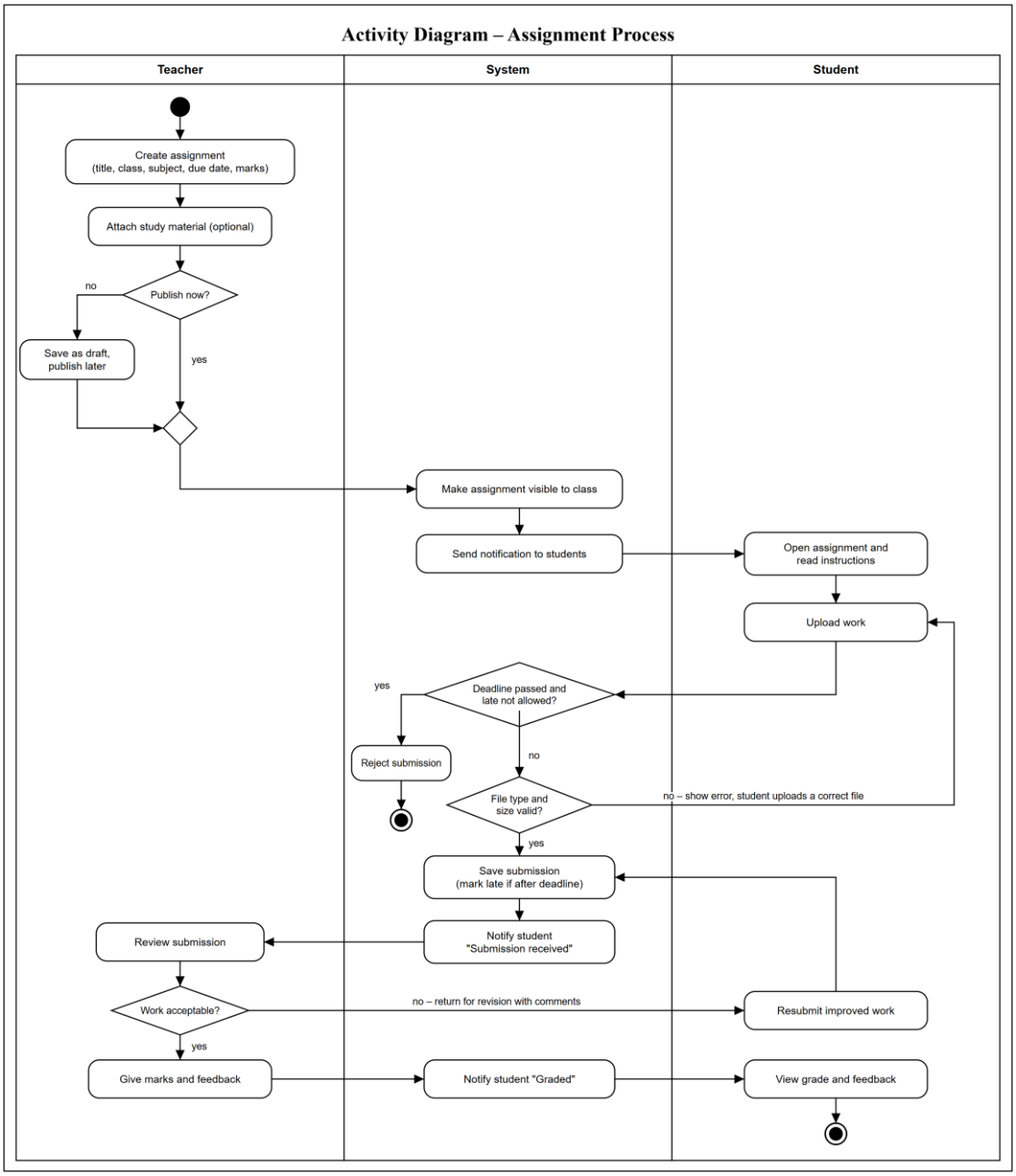


Figure 3: EduFlow – Activity Diagram

Description

This diagram shows the complete assignment process with three swimlanes: Teacher, System and Student. The teacher creates an assignment (title, class, subject, due date, marks) and optionally attaches study material. If it is not published now it is saved as a draft and published later; both branches join at a merge node. The System then makes the assignment visible to the class and sends a notification to the students.

The student opens the assignment, reads the instructions and uploads work. The System first checks whether the deadline has passed and late submission is not allowed; if so the submission is rejected and the flow ends. Otherwise the System checks the file type and size. An invalid file

shows an error and the student uploads a correct file again. A valid submission is saved (marked late if it is after the deadline) and the student is notified that it was received.

The teacher reviews the submission. If the work is acceptable the teacher gives marks and feedback, the System notifies the student that it is graded, and the student views the grade and feedback. If not, the teacher returns it for revision with comments; the student resubmits improved work, which is saved again and comes back to the teacher for review.

Key Elements

- **Swimlanes:** Teacher, System and Student show who performs each action.
- **Actions:** Create assignment, attach study material, save as draft, make visible, send notification, open assignment, upload work, reject submission, save submission, notify, review, give marks and feedback, resubmit improved work, view grade and feedback.
- **Decision nodes:** Publish now?, Deadline passed and late not allowed?, File type and size valid? and Work acceptable?
- **Merge node:** The draft and publish-now branches join before the assignment is made visible.
- **Loops:** An invalid file returns to Upload work, and a returned submission goes back through Save submission to Review submission.
- **Initial and final nodes:** One start node and two end nodes: rejected submission, and grade viewed by the student.

4. Use Case Diagram

A use case diagram shows the functions (use cases) a system provides and the actors who interact with them.

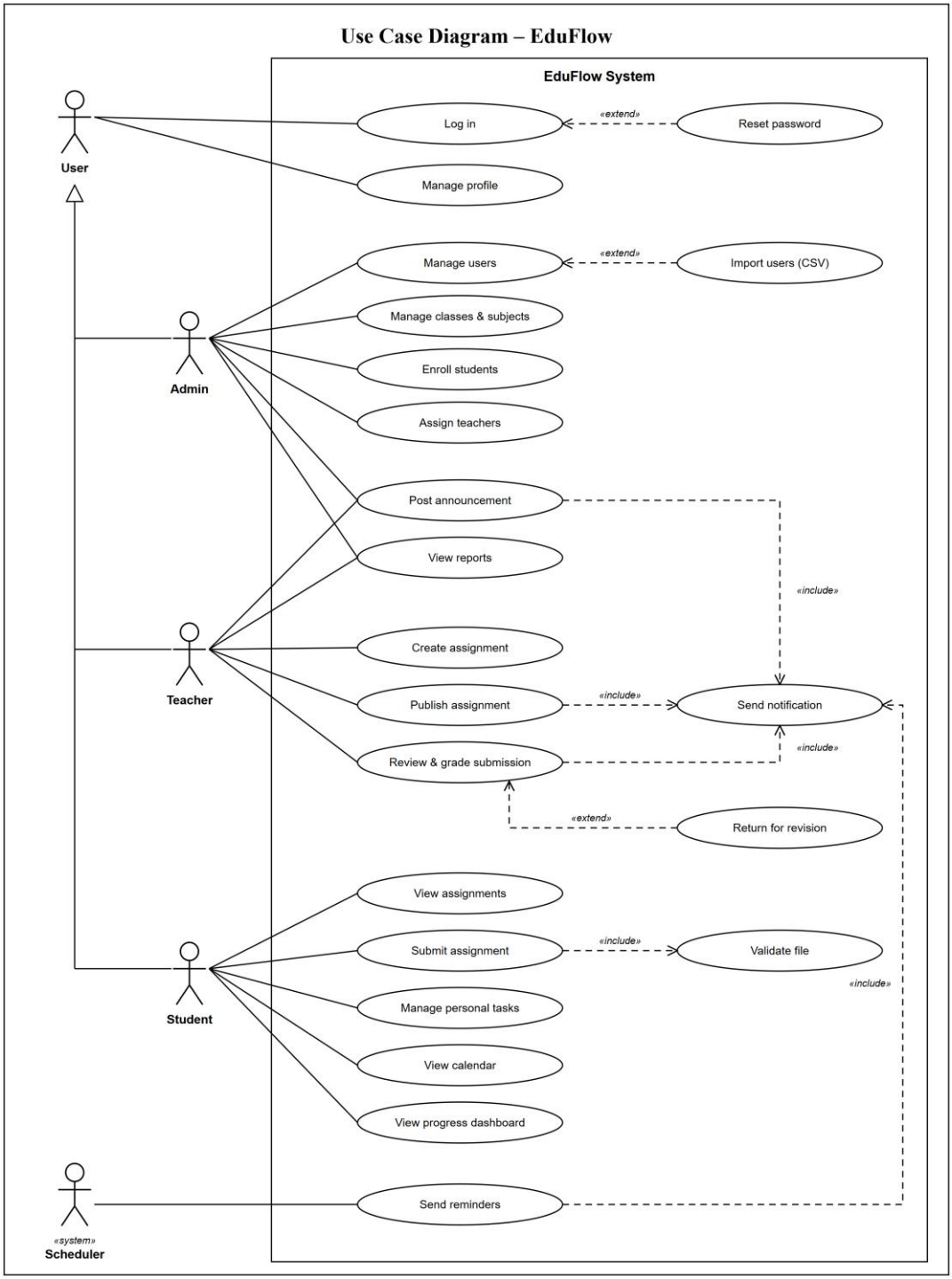


Figure 4: EduFlow – Use Case Diagram

Description

The system boundary is the EduFlow System. The general actor User can Log in and Manage profile; Reset password extends Log in. Admin, Teacher and Student are specialisations of User (actor generalisation), so all three inherit these use cases.

The Admin manages users, manages classes & subjects, enrolls students, assigns teachers, posts announcements and views reports; Import users (CSV) extends Manage users. The Teacher posts announcements, views reports, creates and publishes assignments and reviews & grades submissions; Return for revision extends Review & grade submission. The Student views assignments, submits assignments (which includes Validate file), manages personal tasks, views the calendar and views the progress dashboard.

The Scheduler (cron job) is a system actor that sends reminders. Post announcement, Publish assignment, Review & grade submission and Send reminders all include Send notification, so every important event reaches the users concerned.

Key Elements

- **Actors:** User (general), Admin, Teacher, Student and Scheduler (cron job, «system»).
- **Actor generalisation:** Admin, Teacher and Student inherit from User (hollow-triangle arrow).
- **System boundary:** The rectangle labelled EduFlow System contains all use cases.
- **Use cases:** Log in, Reset password, Manage profile, Manage users, Import users (CSV), Manage classes & subjects, Enroll students, Assign teachers, View reports, Create assignment, Publish assignment, Review & grade submission, Return for revision, Post announcement, View assignments, Submit assignment, Validate file, Manage personal tasks, View calendar, View progress dashboard, Send notification, Send reminders.
- **Include relationships:** Submit assignment includes Validate file; Publish assignment, Review & grade submission, Post announcement and Send reminders include Send notification.
- **Extend relationships:** Reset password extends Log in; Import users (CSV) extends Manage users; Return for revision extends Review & grade submission.

5. Component Diagram

A component diagram shows how a system is divided into modular components and the interfaces through which they depend on each other.

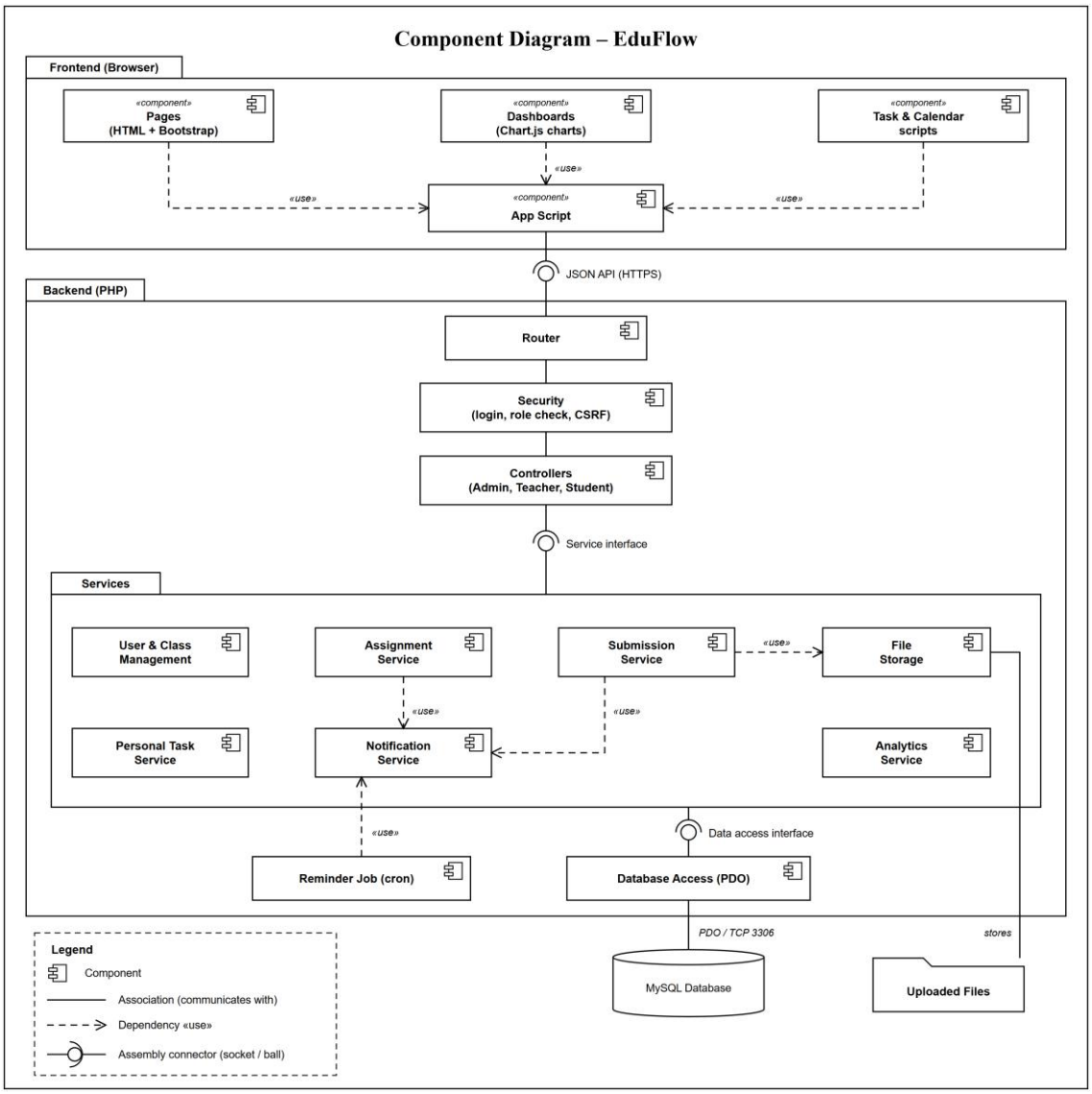


Figure 5: EduFlow – Component Diagram

Description

The diagram uses three kinds of relationship: dependencies (dashed «use» arrows), associations (solid lines) and assembly connectors (ball-and-socket), as shown in the legend. In the Frontend (Browser) package, the Pages (HTML + Bootstrap), the Dashboards (Chart.js charts) and the Task & Calendar scripts each have a dependency on the App Script component, which makes the API calls, handles forms and renders tables.

The App Script is joined to the backend Router by an assembly connector: the App Script requires the JSON API (HTTPS) interface (socket) and the Router provides it (ball). Inside the Backend (PHP) package the Router, Security (login, role check, CSRF) and Controllers (Admin, Teacher, Student) are linked by associations. A second assembly connector joins the Controllers, which require the Service interface, to the Services package that provides it.

Inside the Services package, the Assignment and Submission services depend on the Notification Service, and the Submission Service depends on File Storage; the Reminder Job (cron) also depends on the Notification Service. A third assembly connector joins the Services package, which requires the Data access interface, to the Database Access (PDO) component. Database Access is associated with the MySQL Database (PDO / TCP 3306), and File Storage is associated with the Uploaded Files folder, where it stores the files.

Key Elements

- **Components:** Pages, Dashboards, Task & Calendar scripts, App Script, Router, Security, Controllers, User & Class Management, Assignment, Submission, Personal Task, Notification and Analytics services, File Storage, Reminder Job, Database Access (PDO), MySQL Database and Uploaded Files.
- **Assembly connectors (ball-and-socket):** JSON API (HTTPS) between App Script and Router; Service interface between Controllers and the Services package; Data access interface between the Services package and Database Access.
- **Associations (solid lines):** Router – Security – Controllers; Database Access – MySQL Database; File Storage – Uploaded Files.
- **Dependencies (dashed «use» arrows):** Pages, Dashboards and Task & Calendar scripts on App Script; Assignment Service, Submission Service and Reminder Job on Notification Service; Submission Service on File Storage.
- **Packages:** Frontend (Browser), Backend (PHP) and the nested Services package group related components.
- **Legend:** Explains the symbols for component, association, dependency and assembly connector.

6. Sequence Diagram

A sequence diagram shows how objects interact over time by exchanging messages in a particular order.

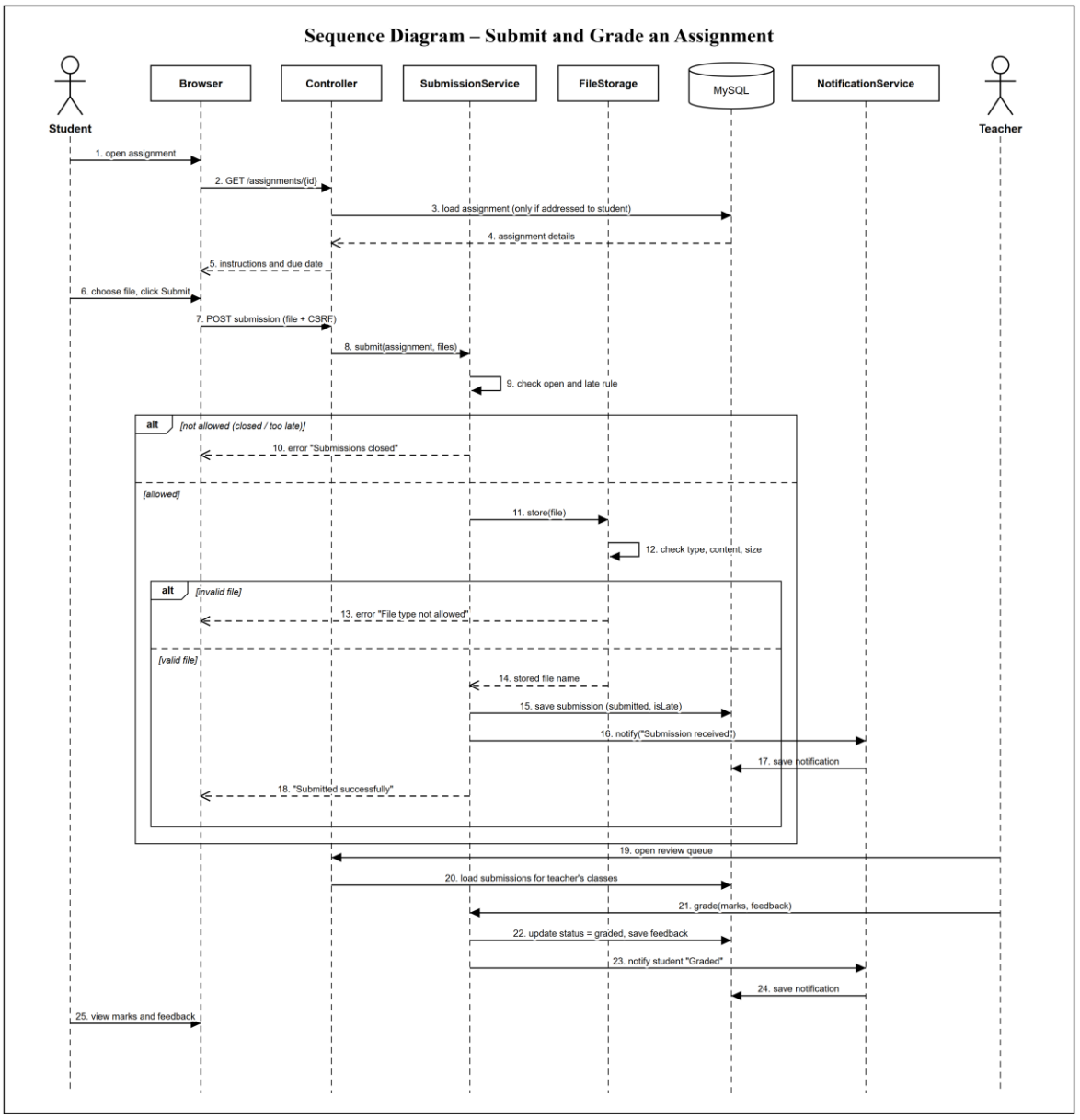


Figure 6: EduFlow – Sequence Diagram

Description

This diagram shows a student submitting an assignment and the teacher grading it. The student opens the assignment; the Browser requests it from the Controller, which loads it from MySQL only if it is addressed to that student, and shows the instructions and due date.

When the student chooses a file and clicks Submit, the Browser posts the file with the CSRF token. The Controller calls SubmissionService.submit(), which checks that the assignment is open and applies the late rule. If submission is not allowed an error is returned. Otherwise FileStorage stores the file after checking its type, real content and size; an invalid file returns an error. For a valid file the submission is saved with status submitted, NotificationService saves a "Submission received" notification, and the Browser shows "Submitted successfully".

The teacher then opens the review queue, which loads the submissions for the teacher's classes, and grades the work. SubmissionService updates the status to graded and saves the feedback, NotificationService notifies the student, and the student views the marks and feedback.

Key Elements

- **Lifelines:** Student and Teacher (actors), Browser, Controller, SubmissionService, FileStorage, MySQL (database) and NotificationService.
- **Messages:** Numbered synchronous calls (solid arrows) and return messages (dashed arrows) in time order.
- **Combined fragments:** alt fragments choose between not allowed / allowed and invalid / valid file.
- **Self-messages:** SubmissionService checks the open/late rule and FileStorage checks the file.
- **Security steps:** the CSRF token is sent with the submission and assignments are loaded only for the students they are addressed to.

7. Deployment Diagram

A deployment diagram shows the physical architecture of a system: the hardware nodes, the execution environments running on them, and the software artifacts deployed on each.

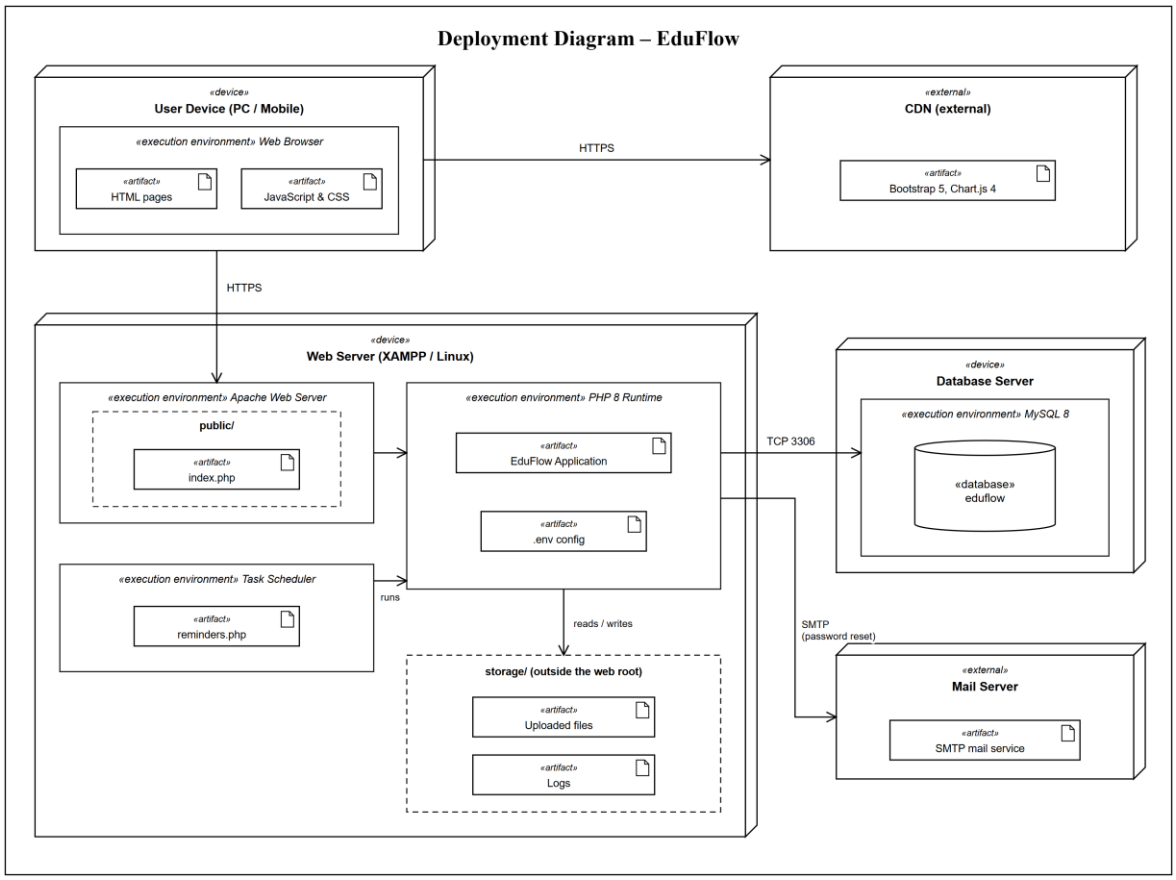


Figure 7: EduFlow – Deployment Diagram

Description

The User Device (PC or mobile) runs a Web Browser execution environment, which holds the HTML pages and the JavaScript & CSS artifacts of the EduFlow user interface. The browser communicates with the Web Server over HTTPS, and loads the Bootstrap 5 and Chart.js 4 libraries from an external CDN, also over HTTPS.

The Web Server (XAMPP on Windows or Linux) contains three execution environments. The Apache Web Server serves only the public/ folder, whose index.php is the single entry point for every request. Apache passes requests to the PHP 8 Runtime, which runs the EduFlow Application artifact and reads its settings from the .env config artifact. The Task Scheduler (cron or Windows Task Scheduler) runs reminders.php, which calls the application to send due-soon and overdue reminders. The application writes uploaded files and logs to the storage/ folder, which is outside public/ and therefore cannot be downloaded directly from the web.

The EduFlow Application connects to the Database Server over TCP port 3306, where the MySQL 8 execution environment stores the eduflow database. For password-reset emails the application uses an external Mail Server over SMTP. Keeping the database on its own node lets it be secured, backed up and scaled independently of the web server.

Key Elements

- **Nodes (devices):** User Device (PC / Mobile), Web Server (XAMPP / Linux) and Database Server, drawn as 3D boxes with the «device» stereotype.
- **Execution environments:** Web Browser, Apache Web Server, PHP 8 Runtime, Task Scheduler and MySQL 8, nested inside their devices with the «execution environment» stereotype.
- **Artifacts:** HTML pages, JavaScript & CSS, index.php, EduFlow Application, .env config, reminders.php, Uploaded files, Logs, the eduflow database and the SMTP mail service.
- **External nodes:** CDN (Bootstrap 5, Chart.js 4) and Mail Server (SMTP mail service), marked «external».
- **Communication paths:** HTTPS between browser and web server and between browser and CDN; TCP 3306 between the application and MySQL; SMTP to the mail server for password reset.
- **Security in deployment:** only public/ is exposed by Apache; source code, .env and storage/ stay outside the web root.